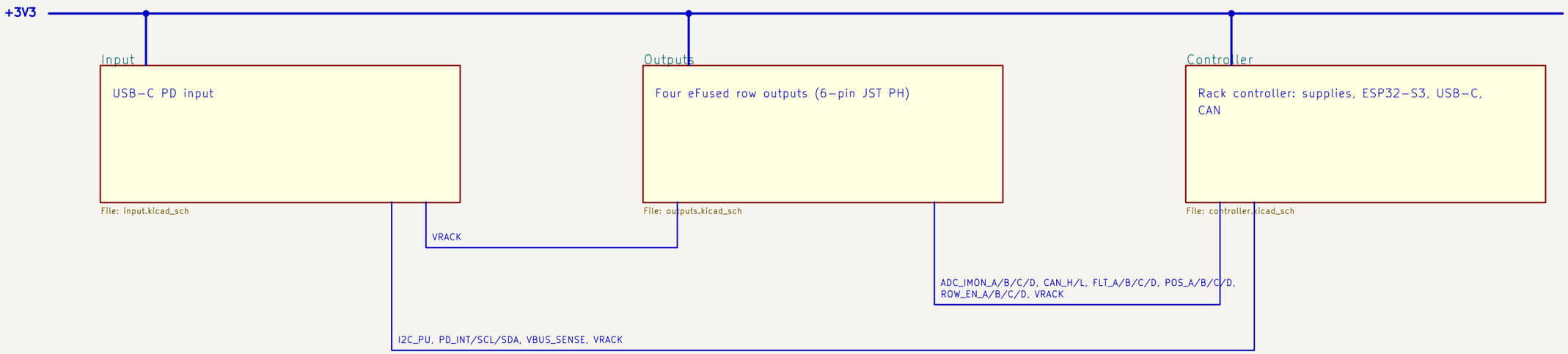



Power hub v1.0 — USB-C PD in (CH224A: 9 V, then 20 V or 28 V EPR as the controller asks) = VRACK → 4 × TPS26600 eFuse (1.6 A) → 6-pin JST PH rows A-D.
Rack controller: ESP32-S3-MINI-1 on VBUS, TJA1462AT (CAN master, star bus, both terminations here), POS token per row.
Rows are OFF without firmware: SHDN is pulled low; the controller turns each row on once the PD contract is final.

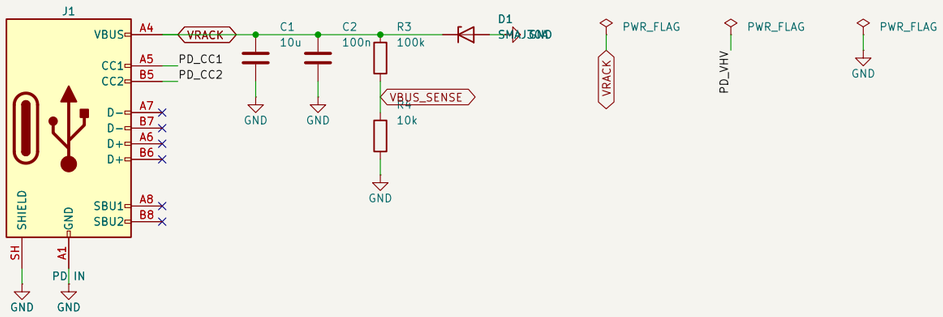
Generated from electronics/power-hub/design.py — edit the Python, then regenerate
USB-C PD distribution + rack controller (ESP32-S3, CAN master) for the hw-testing-rack
**hw-testing-rack**

Sheet: /
File: power-hub.kicad_sch

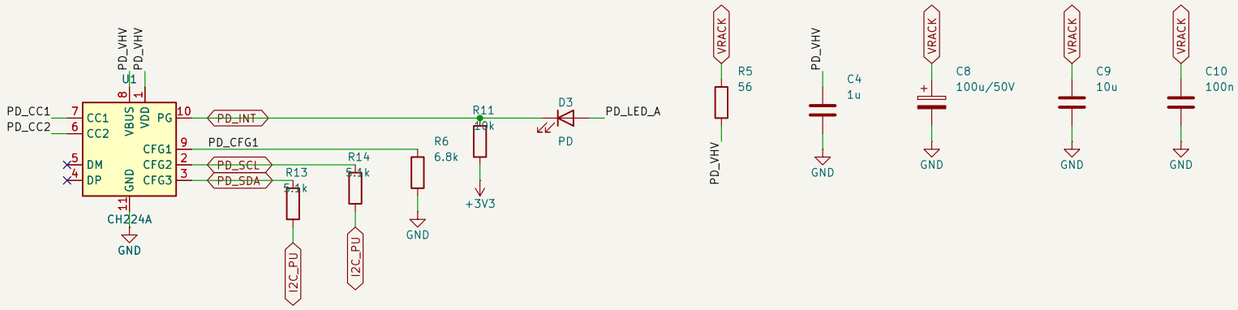
***Title: Power hub — Overview***

| | | |
|---|---|---|
| Size: A3 | Date: | **Rev: 1.0** |
| KiCad E.D.A. 10.0.6 | | Id: 1/4 |

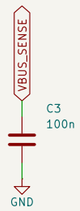
## USB-C PD input (5 A receptacle, CC to the sink controller), TVS; VBUS is VRACK (no input switch)



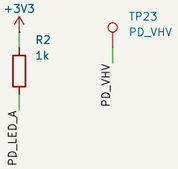

## CH224A USB PD 3.2 sink: 9 V at plug-in (CFG1 6.8k), then 20 V / 28 V EPR as the controller asks over I2C




## I2C to the controller, VBUS sense




## PD status (front LED): on while the CH224A reports power good (PG low)




U1 is the CH224A (WCH, PD 3.2): CFG1 6.8k asks for 9 V at plug-in and turns its I2C on (0x22); the controller writes 0x0A: 5 = 28 V EPR when offered, else 4 = 20 V (single-byte I2C transfers). CH224 V2.1 lists the source's PDOs (0x60-0x8F) for both parts, readable under 100 W (p. 7): with our 100 W charger, still to check (schematic review R3); without them the firmware tries 28 V and checks VBUS.
No input switch: VBUS is VRACK. The eFuses (UV 15 V, off until enabled, reverse current blocked) are the switches; the controller turns the rows on once 0x09 says PD (and EPR) and VBUS_SENSE reads the voltage.
SMAJ30A: 30 V stand-off, above a 28 V EPR contract; the eFuses (60 V) and the TPS26600 OVP (31-34 V) follow.
R5/C4: the CH224A's VHV/VBUS pins are rated 32 V; the RC filters the fast edges the TVS can't reach.
VBUS_SENSE: 100k/10k from VBUS → 2.5 V at 28 V for the ESP32's ADC; also tells the firmware when to enable I2C_PU.

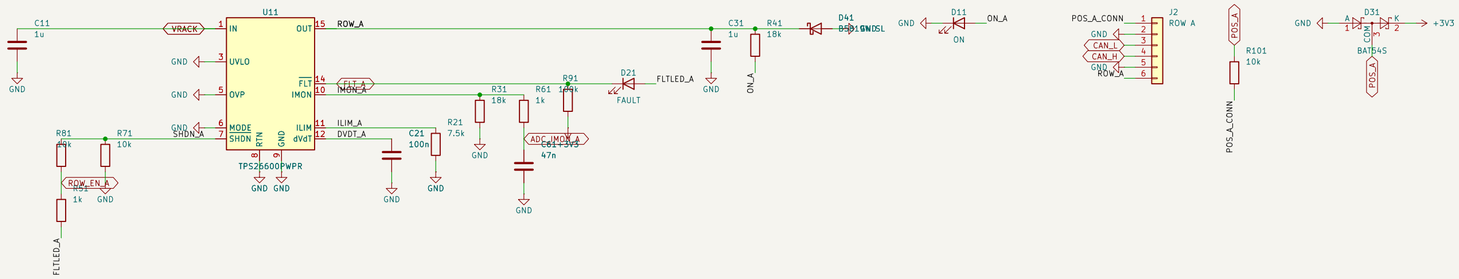
Generated from electronics/power-hub/design.py — edit the Python, then regenerate
USB-C PD distribution + rack controller (ESP32-S3, CAN master) for the hw-testing-rack
**hw-testing-rack**

Sheet: /Input/
File: input.kicad_sch

***Title: Power hub — USB-C PD input***

| Size: A3 | Date: | **Rev: 1.0** |
|---|---|---|
| KiCad E.D.A. 10.0.6 | | Id: 2/4 |

## Row A: TPS26600 eFuse 1.6 A, enable, ON / FAULT LEDs, 6-pin JST PH

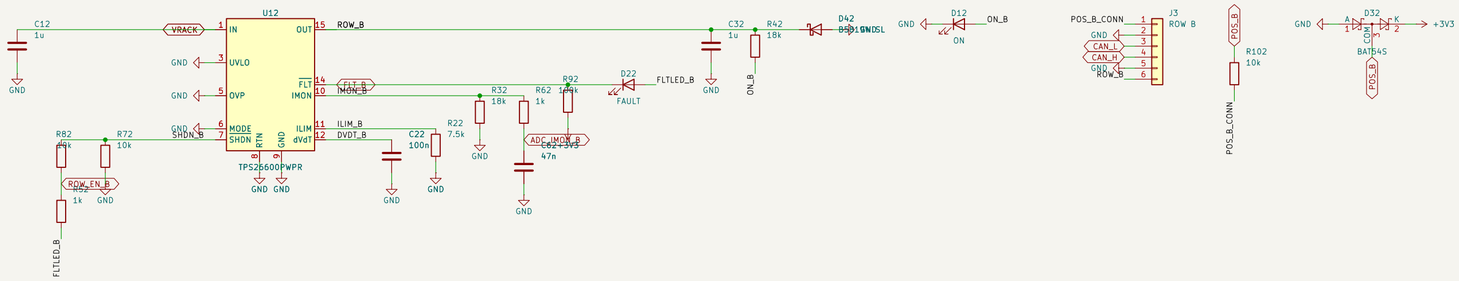









## Row B: TPS26600 eFuse 1.6 A, enable, ON / FAULT LEDs, 6-pin JST PH

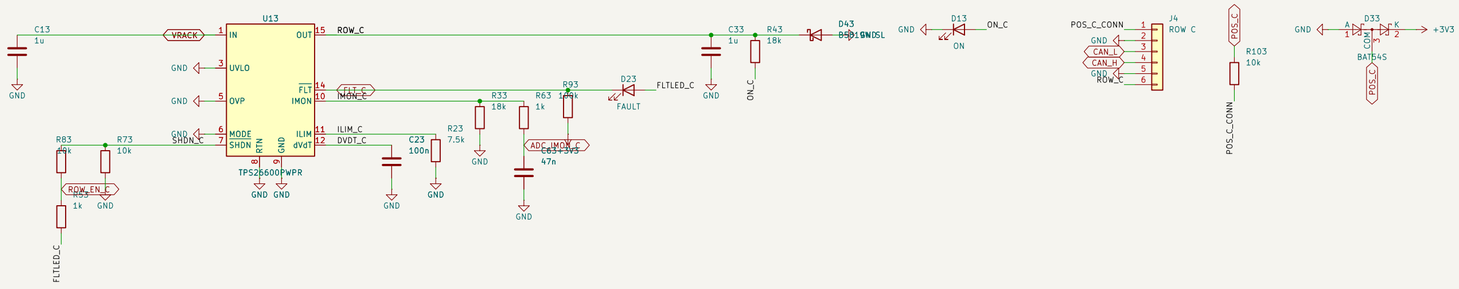









## Row C: TPS26600 eFuse 1.6 A, enable, ON / FAULT LEDs, 6-pin JST PH

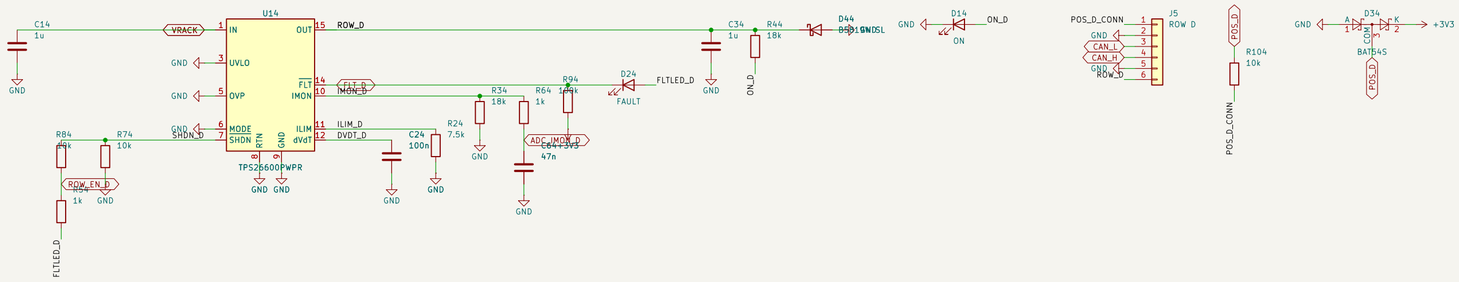






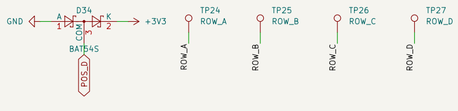



## Row D: TPS26600 eFuse 1.6 A, enable, ON / FAULT LEDs, 6-pin JST PH










TPS26600: I_LIM ≈ 12 A·kΩ / R_ILIM → 7.5k ≈ 1.6 A (JST PH: 2 A per contact).
t_dVdT = 8e3 · V_IN · C_dVdT → 100 nF ≈ 16 ms at 20 V, 22 ms at 28 V. A row of four nodes (≈ 450 µF) then
charges at C / (8e3 · C_dVdT) ≈ 0.56 A (≤ 1.3 A at the corners with the nodes' load), under the 1.6 A limit.
The eFuse takes ≈ 7-9 W typically, ≈ 14 W worst, for 19-24 ms: TI Fig. 55 with its 30 % margin (47 nF: ≈ 21 W, 12 ms).
IMON: 78 µA/A into 18k → 1.4 V/A (2.2 V at the limit). 1k + 47 nF to the ESP32's ADC: the pin is a current
source, so τ = (18k + 1k) × 47 nF ≈ 0.9 ms, against the firmware's 250 ms polling.
SHDN: 10k to GND (row off: ≤ 10 µA from the pin, 0.1 V). ROW_EN through 10k → 1.65 V (TI: ≥ 1 V to turn on, p. 26).
The firmware holds ROW_EN with the ESP32's GPIO hold, so a controller reset leaves the rows as they were.
FLT: open drain, 100k to 3.3 V for the controller; the red LED (1k) is fed from ROW_EN: it lights only for
a fault on an enabled row (FLT is also low while a row is off or starting).
D41-D44 B5819W: GND → each row's output at its connector, for the cable's kick when the eFuse cuts a short.
UVLO / OVP / MODE to GND: internal 15 V / 31-34 V (typ 32.6 V) thresholds, current-limit mode, auto-retry.
RTN to GND: the eFuses' own reverse-polarity block unused (TI allows it): VRACK is the PD source's output.

Generated from electronics/power-hub/design.py — edit the Python, then regenerate
USB-C PD distribution + rack controller (ESP32-S3, CAN master) for the hw-testing-rack

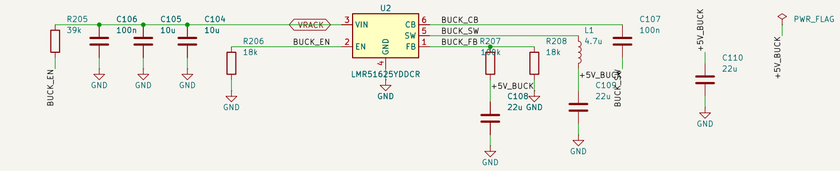
**hw-testing-rack**

Sheet: /Outputs/
File: outputs.kicad_sch

**Title: Power hub — Four eFused row outputs (6-pin JST PH)**

| Size: A2 | Date: | **Rev: 1.0** |
|---|---|---|
| KiCad E.D.A. 10.0.6 | | Id: 3/4 |

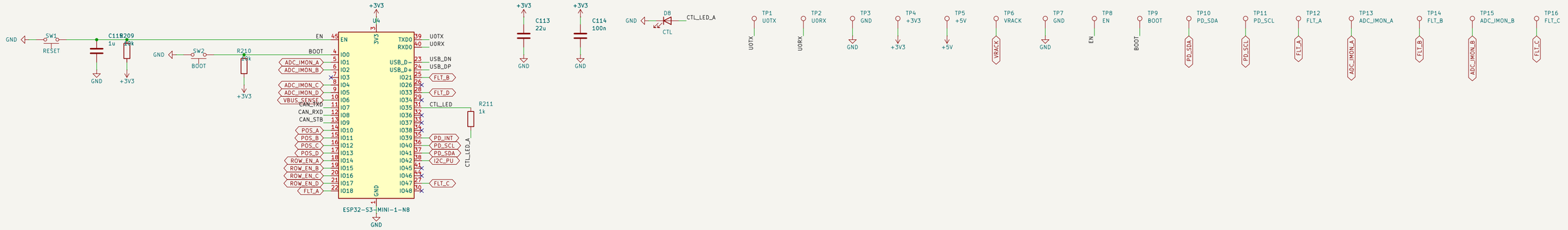
## 5 V buck: LMR51625 (2.5 A; the controller takes < 0.7 A) and 4.7 µH, as the bay node (TI SLUSF64B table 8-1), from VRACK

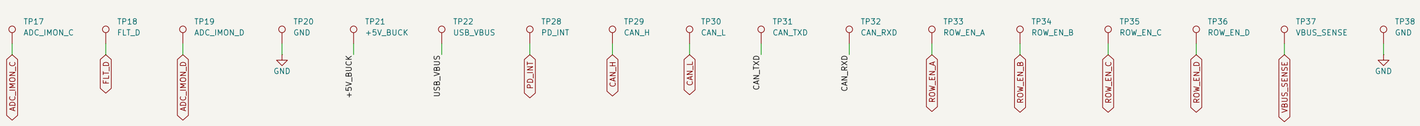



## 5 V for the controller: the buck or the console USB-C (diode OR), 3.3 V low-dropout LDO

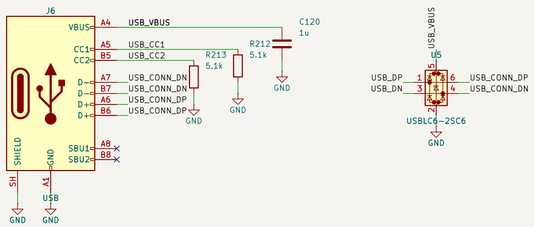



## ESP32-S3-MINI-1 (antenna at the board's rear edge: no copper under it, either layer)

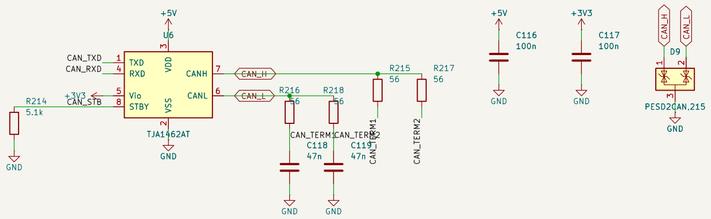



## USB-C: the controller's console / firmware (native USB), and its 5 V without a PD charger




## CAN: TJA1462AT, star bus (all row ports in parallel), both terminations here




## Mounting (M3, on the power box's standoffs)




Generated from electronics/power-hub/design.py — edit the Python, then regenerate
USB-C PD distribution + rack controller (ESP32-S3, CAN master) for the hw-testing-rack

**hw-testing-rack**

Sheet: /Controller/
File: controller.kicad_sch

***Title: Power hub — Rack controller: supplies, ESP32-S3, USB-C, CAN***

| Size: A2 | Date: | **Rev: 1.0** |
|---|---|---|
| KiCad E.D.A. 10.0.6 | | Id: 4/4 |